

GÖREV 3: Clean Code & SOLID Prensipleri Teşhisi (25 Puan)

Aşağıda junior bir geliştirici tarafından yazılmış temsili bir sipariş sınıfı yer almaktadır:

```
class KahveSiparisYoneticisi {  
    void sepetHesaplaVeIndirimUygula() { ... }  
    void krediKartindanTahsilatYap() { ... }  
    void siparisiVeritabaninaKaydet() { ... }  
    void musteriYiSmsIleBilgilendir() { ... }  
  
    double indirimHesapla(String musteriTipi, double tutar){  
        if (musteriTipi == "OGRENCI") return tutar * 0.80;  
        else if (musteriTipi == "OGRETMEN")  
            return tutar * 0.85;  
        else return tutar; }  
}
```

Soru:

1. Bu sınıfta Single Responsibility Principle (SRP - Tek Sorumluluk) nasıl ihlal edilmiştir? Sınıfı hangi küçük parçalara bölmeliyiz? (Kod yazmanıza gerek yoktur, 2 cümleyle açıklayın).
2. indirimHesapla fonksiyonunda yarın yeni bir müşteri tipi (örneğin "DOKTOR") geldiğinde if-else kodunu değiştirmek zorunda kalmak hangi SOLID prensibine aykırıdır? (Open/Closed Principle - OCP).

Cevap-1:

SRP ihlalinin olabilmesi için adından mütevellit Single Responsibility Principle yani bir class yapısına birden fazla birbirinden farklı sorumluluklar ve işler vermeniz gerekir.

KahveSiparisYoneticisi aynı anda indirim, kredi kartı çekimi, veri tabanı işlemi ve sms gönderimini üstlenmektedir. SRP'nin ihlalinde bir kodu güncellemek için kendisiyle alakalı kod yapısına dokunduğumuzda aynı zamanda iç içe geçmiş birçok kod yapısının bulunduğu programa da müdahale etmek zorunda kalırız, oysaki ayrı class lar da her işlem kendi bünyesinde değişikliğe gittiğinden kodun yapısında yapılan değişiklikler daha kolay gerçekleştirilmiş olacaktır.

Sınıfı küçük parçalara şu şekilde böleriz tek bir class da kullanmak yerine 4 ayrı class'lara böleriz. Artık KahveSiparisYoneticisi yerine SepetHesabı, KrediKartıTahsilatı, VeriTabanıKayıt, SmsBilgilendirme şeklinde bölebiliriz. Kodun alakada olduğu bölümleri de revize ederiz.

Cevap-2:

Soruda da değinildiği gibi OCP yani Open/Closed Principle'ına aykırı bir durum vardır ortada. Çünkü yeni müşteri tipi gelmesi demek koda ekleme yapmak demektir oysaki biz OCP ile programın değişikliğe kapalı genişletilmeye açık olmasını isteriz oysaki DOKTOR'u eklemek demek if-else yapısını değiştirip araya yeni bir kod ekleyip kodu değiştirmek demektir.